

TextHack> find "knowledge"

TextHack

An advanced algorithm engine for enterprise knowledge search

Enterprise Knowledge Intelligence Platform · DSA-3 Review

$O(n+m)$

$O(n+z)$

$O(n \log n)$

$O(n \cdot m)$

$O(V^2E)$

Team

M. Sathya Krishna 2510080006

Yashwanth 2510080001

Aneeq 2510080005

Abhinav 2510080002

Guide

Dr. P. Anitha

Searching enterprise knowledge is a string problem

Background

- Organisations keep their knowledge in thousands of documents: policies, handbooks, reports, research.
- Finding the right one depends on text search that copes with how people actually type.
- Real queries contain typos, reordered words and several terms at once, and documents cite one another.

Need for the project

- Database LIKE / ILIKE matches exact substrings only; one typo returns nothing.
- Scanning once per search term costs $O(k \cdot n)$ for k terms.
- Libraries hide cost and correctness; the course requires building and proving the algorithms.
- How strongly two documents are connected through citations is a graph problem.

Three kinds of query the engine must answer

fuzzy "recieve" ~2

a typo

findall "vpn" "laptop" "policy"

many terms at once

doc 0 → ... → doc 5

linked documents

What we set out to solve

Build, from first principles and without library shortcuts, a verified engine of string, dynamic-programming, graph, approximation and randomized algorithms, and use it to make enterprise document search tolerant of typos, fast for many terms at once, and able to analyse citation structure, with the correctness and complexity of every algorithm demonstrated.

$O(n+m) \cdot O(n+z)$

Exact and multi-pattern matching

Find every occurrence of one or many terms in linear time, with no worst-case input.

$O(n \cdot m)$

Approximate matching

Rank near-misses such as "recieve" for "receive", and align two texts to show where they agree.

$\text{max flow} = \text{min cut}$

Structure

Measure how many independent citation chains connect two documents, and which citations every chain depends on.

Goals and expected outcomes

1

Implement 20 algorithms from scratch

No java.util in algorithm code

31 files · 4,287 lines

2

Prove correctness with tests

Known cases, edge cases, randomised cross-checks

452 assertions · 0 failures

3

Document and measure complexity

Registry, complexity reference and benchmarks

22 registry entries

4

One engine API and query language

find · findall · fuzzy · similar · prime

TextHack facade

5

Improve the platform's search

Keyword and fuzzy scoring in the search backend

KMP · Aho-Corasick · OSA

6

Let users run the algorithms

REST API and a TextHack workbench page

4 endpoints · 4 panels

The classic algorithms we built on

Problem	Published work	What it gives the engine
Exact matching	Knuth, Morris & Pratt (1977); Karp & Rabin (1987); Gusfield, Z-algorithm (1997)	Linear-time search; Rabin-Karp must verify each hash hit
Multi-pattern matching	Aho & Corasick (1975)	All k patterns in one pass, $O(n + z)$
Text indexing	Manber & Myers, suffix arrays (1993); Kasai et al., LCP (2001)	Repeated substrings; $O(n \log n)$ build, $O(n)$ LCP
Edit distance and alignment	Levenshtein (1966); Damerau (1964); Needleman & Wunsch (1970); Smith & Waterman (1981)	Typo tolerance and global / local alignment, $O(n \cdot m)$
Network flow	Ford & Fulkerson (1956); Edmonds & Karp (1972); Dinic (1970); Kuhn (1955)	Citation influence as max flow, bottleneck as min cut
Approximation and randomization	Graham (1966, 1969); Miller (1976), Rabin (1980); Carter & Wegman (1979); Vitter (1985)	Scheduling bounds, primality, universal hashing, sampling

Existing approaches and their limitations

Database pattern matching

SQL LIKE / ILIKE in PostgreSQL

- Exact substrings only: "recieve" never finds "receive"
- Reordered words do not match
- No relevance ranking

Full-text search engines

Apache Lucene / Elasticsearch

- Fuzzy search exists, but as a large external service
- Algorithms are a black box to the application
- Does not meet the course rule of building algorithms from scratch

Naive textbook code

Typical classroom implementations

- Z-algorithm with a sentinel breaks on text containing it
- Rabin-Karp reporting unverified hash hits
- $(a \cdot b) \bmod m$ overflowing 64 bits; Fermat tests fooled by Carmichael numbers

Our approach: build each algorithm ourselves, check it against a simple reference, and run the same verified code in the product.

TextHack: one verified engine, used by the whole platform

A dependency-free Java engine of 20 algorithms in five families, behind one facade and a small query language. The platform's Spring Boot backend compiles the same source for keyword and fuzzy search, and a workbench page lets users run the algorithms on their own input.

$O(n+m)$

Exact search

KMP, Z-algorithm and Rabin-Karp, with no pathological input for KMP and Z

$O(n+z)$

Many terms in one pass

Aho-Corasick automaton with failure and output links

$O(n \cdot m)$

Typo tolerance

Levenshtein and Damerau-Levenshtein (restricted OSA and unrestricted)

$O(n \cdot m)$

Alignment

Needleman-Wunsch (global) and Smith-Waterman (local)

$O(V^2E)$

Citation analysis

Dinic max flow and the Edmonds-Karp minimum cut

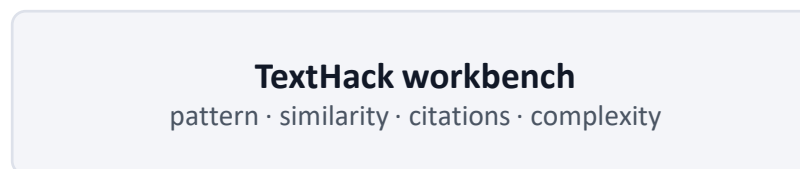
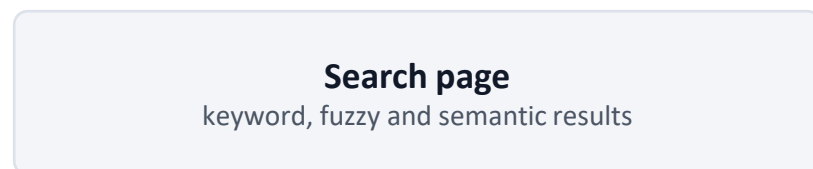
ratio bounds

Provable guarantees

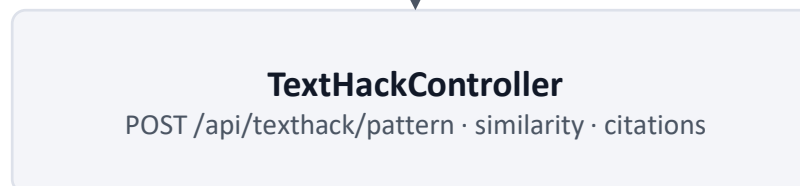
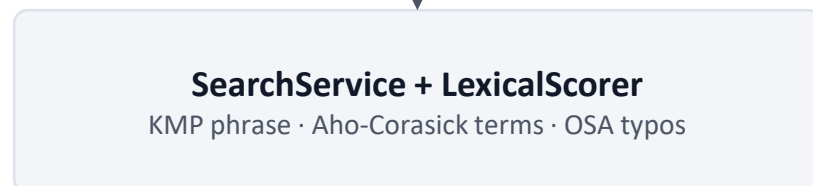
Miller-Rabin exact for 64-bit, LPT within $4/3$, vertex cover within $2\times$

Where TextHack sits in the platform

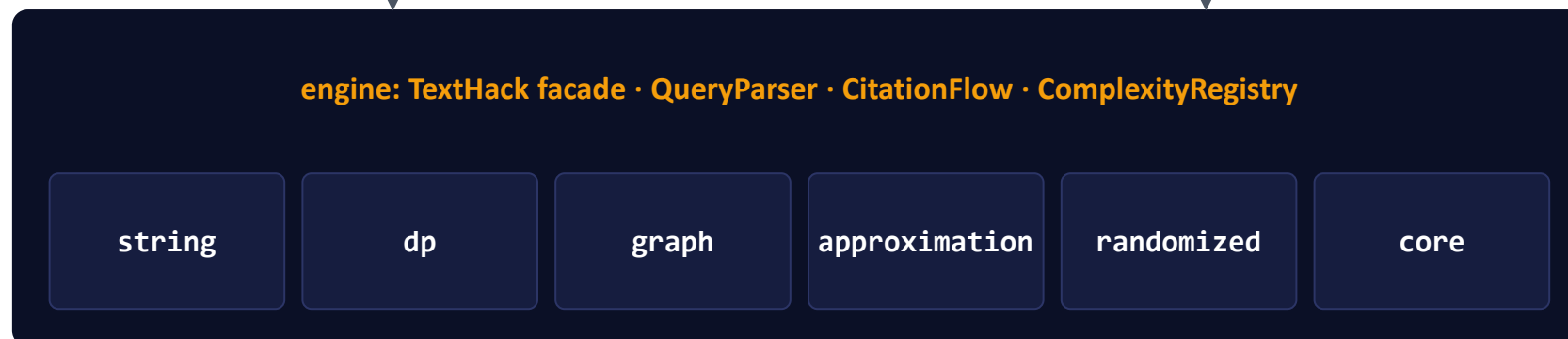
React frontend



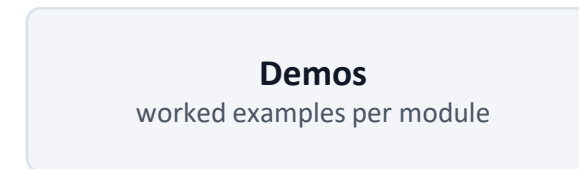
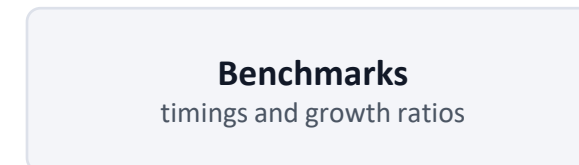
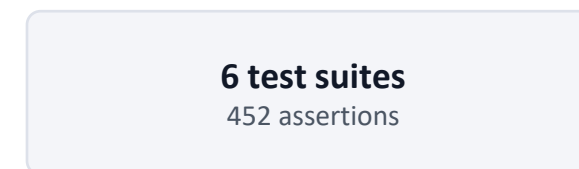
Spring Boot backend



TextHack engine (DSA-3, pure Java, no dependencies)



Verification (runs without the platform)



compiled together by run-tests.sh with
javac -Xlint:all

*Search data: document titles and descriptions
come from PostgreSQL; MongoDB and Qdrant
serve content and semantic vectors.*

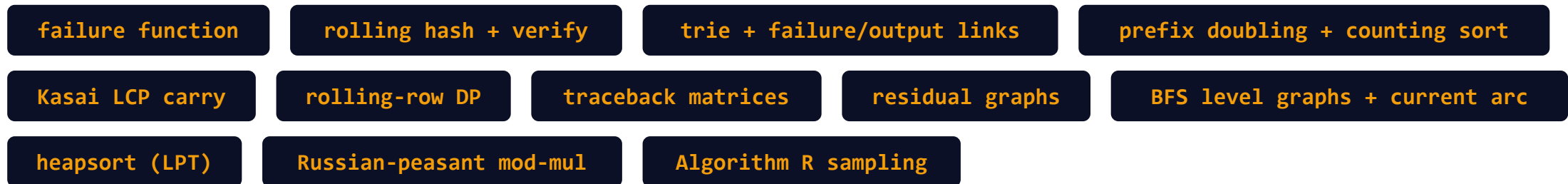
Component description

Package	Responsibility	Main classes
core	Primitives replacing java.util: growable int list, sorted child map, seeded PRNG, matcher contract	IntList, CharMap, Prng, StringMatcher
string	Exact, multi-pattern and suffix-structure algorithms	NaiveSearch, KmpSearch, ZSearch, RabinKarpSearch, AhoCorasick, SuffixArray, LcpArray
dp	Edit distance and sequence alignment	Levenshtein, DamerauLevenshtein, NeedlemanWunsch, SmithWaterman, Alignment
graph	Residual networks, max flow and matching	FlowNetwork, FordFulkerson, EdmondsKarp, Dinic, BipartiteMatching
approximation	NP-hard problems with proven ratios	VertexCover, MakespanScheduling
randomized	Primality, hashing and sampling	MillerRabin, UniversalHashing, ReservoirSampling
engine	One facade, query language, citation analysis, complexity registry	TextHack, QueryParser, Query, CitationFlow, ComplexityRegistry
platform	Search scoring, REST API and workbench page	LexicalScorer, SearchService, TextHackController, TextHackPage

How every algorithm was built and verified



Techniques used



The 20 algorithms, in five families

String

Naive search

$O(n \cdot m)$

KMP

$O(n+m)$

Z-algorithm

$O(n+m)$

Rabin-Karp

$O(n+m)$ exp.

Aho-Corasick

$O(n+z)$

Suffix array

$O(n \log n)$

LCP (Kasai)

$O(n)$

Dynamic programming

Levenshtein

$O(n \cdot m)$

Damerau-Lev. (+OSA)

$O(n \cdot m)$

Needleman-Wunsch

$O(n \cdot m)$

Smith-Waterman

$O(n \cdot m)$

Network flow

Ford-Fulkerson

$O(E \cdot f)$

Edmonds-Karp

$O(V \cdot E^2)$

Dinic

$O(V^2 \cdot E)$

Bipartite matching

$O(V \cdot E)$

Approximation

Vertex cover

ratio 2

List scheduling

$2 - 1/m$

LPT scheduling

$4/3 - 1/3m$

Randomized

Miller-Rabin

$O(k \log^3 n)$

Universal hashing

$O(1)$

Reservoir sampling

$O(n)$

Languages, tools and requirements

Languages and tools

Java 21

Engine, tests, benchmarks; also verified on Java 25

javac + sh

run-tests.sh compiles with -Xlint:all; no Maven or JUnit

Spring Boot 3.4

Backend compiles the engine as a second source root

React 19 + Vite

TextHack workbench and search pages

Git · GitHub Actions

Version control; CI runs the DSA-3 suites

Docker

Runs the full platform with its databases

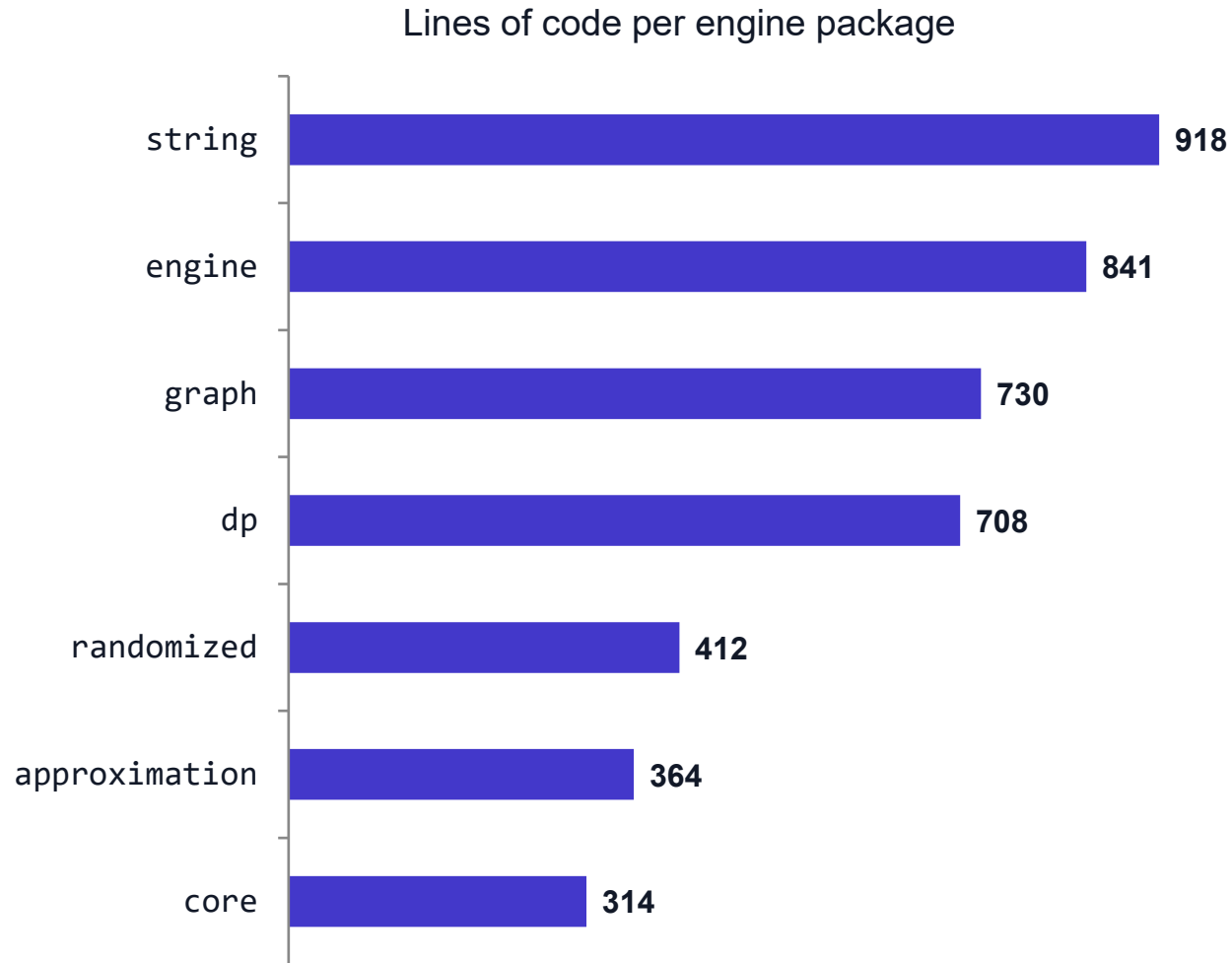
Software requirements

- Engine: any JDK 21 or later; Windows, Linux or macOS
- Full platform: Docker, Node.js 20+, a modern browser

Hardware requirements

- Engine: any dual-core machine with 4 GB RAM
- Benchmarks in this review: AMD Ryzen 7 260 laptop, Windows 11

Modules developed

**31**

engine source files

4,287

lines of algorithm code

3,113

lines of tests (6 suites)

0

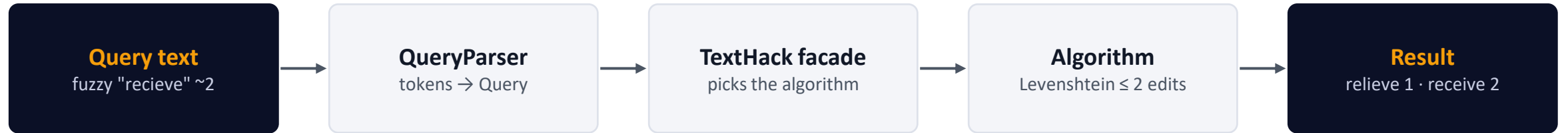
java.util imports in algorithms

0

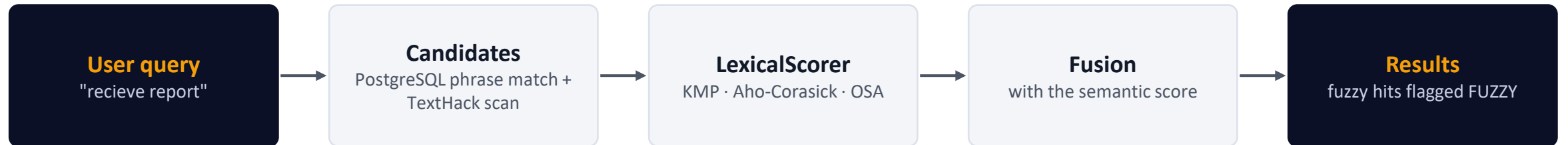
javac -Xlint:all warnings

Working process

Engine query flow



Search scoring flow in the platform



Exact phrase in title (KMP)

1.0

Phrase in description

0.75

Term coverage (Aho-Corasick)

 ≤ 0.9

Term one edit away (OSA)

0.7 credit

Term two edits away

0.5 credit

Terms of ≤ 3 characters

never fuzzed

Workbench: pattern search

The screenshot shows the TextHack workbench interface. On the left is a sidebar with navigation links: Dashboard, Search, Repository, TextHack (selected), and Administration. The main area has a search bar at the top and a user profile 'EIP Administrator'. Below the search bar, the 'TextHack workbench' section is active, showing tabs for Pattern search, Similarity, Citation flow, and Complexity. The 'Pattern search' tab is selected, displaying a text input field with the sentence 'she sells sea shells by the sea shore; he hears the ushers hush.' and a list of patterns: 'he', 'she', and 'hers'. A 'Find matches' button is present. Below the button, the results for the Aho-Corasick algorithm are shown, indicating 11 matches. The matches are displayed as a list of substrings with their starting and ending indices: 'she' at 0-3, 'he' at 1-3, 'she' at 14-17, 'he' at 15-17, 'he' at 25-27, 'he' at 39-41, 'he' at 42-44, 'he' at 49-51, 'she' at 53-56, 'he' at 54-56, and 'hers' at 54-58. The longest repeated substring is identified as ' sea sh'.

11 matches, one pass

Aho-Corasick finds he, she and hers together

Nested patterns

"he" inside "she", "hers" inside "ushers", found via output links

Longest repeated substring

" sea sh", from the suffix array and LCP array

One pattern → KMP

The API switches algorithm by pattern count

Workbench: similarity and citation flow

Similarity and alignment

Edit distance by Levenshtein and Damerau-Levenshtein, which counts a swapped pair as one edit. Needleman-Wunsch aligns the whole of both texts; Smith-Waterman finds the best matching region.

First text:

Second text:

Compare

Levenshtein	3 edits
Damerau	2 edits
Similarity	84%

Global

Score 12 · identity 84% · first 0–18, second 0–19

Local

Score 30 · identity 89% · first 0–18, second 0–18

- Levenshtein 3 edits vs Damerau 2: the swapped "ie" is one edit
- Global (Needleman-Wunsch) and local (Smith-Waterman) alignment shown with scores

Citation flow

Documents are numbered from 0, and each citation points from one document to another. Dinic's maximum flow counts the separate citation chains from source to sink. Edmonds-Karp's minimum cut shows the citations every chain depends on.

Documents: Source: Sink:

Citations, one "from to" per line

Analyse

Influence 2: 2 separate citation chains from 0 to 5.

Bottleneck citations (the minimum cut):

0 → 1 0 → 2

Source side of the cut: 0

- Influence 2: two separate citation chains (Dinic max flow)
- Bottleneck citations 0→1 and 0→2 (Edmonds-Karp min cut)

Complexity registry and Miller-Rabin

Complexity				
Algorithm	Category	Time	Space	Note
Naive search	String	$O(n*m)$	$O(1)$	Reference implementation
KMP	String	$O(n+m)$	$O(m)$	No pathological input
Z-Algorithm	String	$O(n+m)$	$O(m)$	No sentinel required
Rabin-Karp	String	$O(n+m)$ expected	$O(1)$	Every hash hit verified
Aho-Corasick	String	$O(n+z)$	$O(m)$	Multi-pattern, one pass
Suffix array	String	$O(n \log n)$	$O(n)$	Prefix doubling, counting sort
LCP (Kasai)	String	$O(n)$	$O(n)$	Requires the suffix array
Levenshtein	DP	$O(n*m)$	$O(\min(n, m))$	Rolling rows
Damerau-Levenshtein	DP	$O(n*m)$	$O(n*m)$	Unrestricted variant
Damerau-Levenshtein OSA	DP	$O(n*m)$	$O(n*m)$	Restricted variant
Needleman-Wunsch	DP	$O(n*m)$	$O(n*m)$	Global alignment
Smith-Waterman	DP	$O(n*m)$	$O(n*m)$	Local alignment
Ford-Fulkerson	Graph	$O(E*maxFlow)$	$O(V)$	DFS augmenting paths
Edmonds-Karp	Graph	$O(V*E^2)$	$O(V)$	BFS augmenting paths
Dinic	Graph	$O(V^2 * E)$	$O(V)$	$O(E * \sqrt{E})$ on unit capacity
Bipartite matching	Graph	$O(V * E)$	$O(V)$	Kuhn's algorithm
Vertex cover	Approximation	$O(V * E)$	$O(V)$	Ratio 2
List scheduling	Approximation	$O(n * m)$	$O(n * m)$	Ratio $2 - 1/m$
LPT scheduling	Approximation	$O(n \log n)$	$O(n * m)$	Ratio $4/3 - 1/3m$
Miller-Rabin	Randomized	$O(k * \log^3 n)$	$O(1)$	Exact for 64-bit
Universal hashing	Randomized	$O(1)$ / $O(L)$	$O(1)$	Collision $\leq 1/m$
Reservoir sampling	Randomized	$O(n)$	$O(k)$	One pass, unknown length

```

Java 25 · TextHack (DSA-3): examples.Demos: Miller-Rabin versus a Fermat test

$ java -cp out examples.Demos (excerpt)
=====
Primality: Miller-Rabin versus a Fermat test
=====
Carmichael numbers satisfy Fermat's little theorem for every
coprime base, so a Fermat test calls them prime.

n      fermat base 2  miller-rabin
561    prime         composite
1105   prime         composite
1729   prime         composite
2465   prime         composite
6601   prime         composite

Miller-Rabin is not fooled: it checks the square roots of 1,
and a composite has non-trivial ones that a prime does not.

2^61 - 1 = 2305843009213693951 is prime
Exact, not probable: the witness set {2,3,...,37} is proven
sufficient across the whole 64-bit range.

```

Carmichael numbers 561, 1105, 1729... pass a Fermat test; Miller-Rabin rejects every one.

Engine demos: fuzzy matching and scheduling

Java 25 · TextHack (DSA-3): examples.Demos: pattern search, fuzzy matching, suffix/LCP

```
$ java -cp out examples.Demos
=====
Pattern search and fuzzy matching
=====
text: the quick brown fox jumps over the lazy dog
exact 'the' at offsets: 0 31

vocabulary: receive believe retrieve deceive relieve
fuzzy query 'recieve' within 2 edits:
  relieve    distance 1  similarity 0.857
  receive    distance 2  similarity 0.714
  believe    distance 2  similarity 0.714
  retrieve    distance 2  similarity 0.750
'relieve' ranks first at distance 1 -- a single l/c
substitution. The intended correction 'receive' is a
transposition, which costs 2 under plain Levenshtein.
Damerau-Levenshtein scores that transposition as 1 and
would rank them level, which is precisely why the two
distances are kept as separate algorithms.

longest repeated substring of:
the rain in spain falls mainly in the plain
-> " in "
found from the largest LCP entry, since adjacent suffixes
in sorted order share the longest prefixes.
```

Java 25 · TextHack (DSA-3): examples.Demos: list scheduling versus LPT

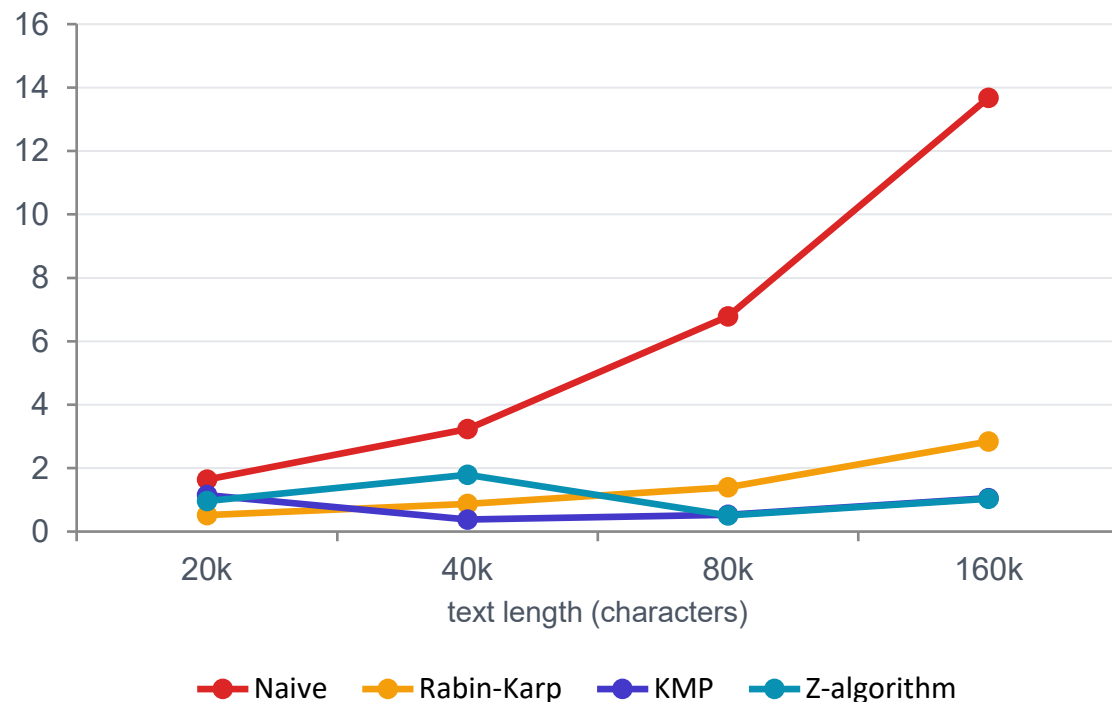
```
$ java -cp out examples.Demos (excerpt)
=====
Scheduling: list order versus LPT
=====
12 jobs of length 1, then one job of length 4, on 4 machines.
Total work is 16, so a perfect split finishes at 4.

list scheduling makespan: 7
  loads: 7 3 3 3
LPT makespan:             4
  loads: 4 4 4 4

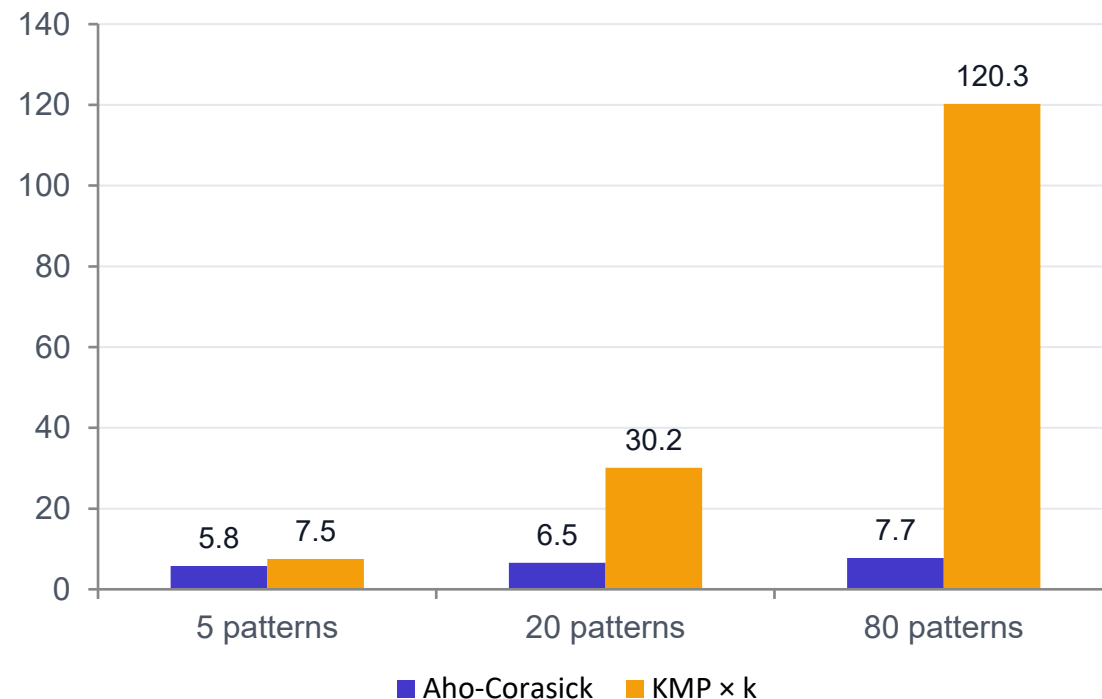
The long job arrives last in list order and lands on an
already-full machine. LPT schedules it first, while every
machine is still empty, and reaches the optimum.
Lower bound: 4
```

The measured growth matches the theory

Exact search time (ms) vs text length



Multi-pattern search time (ms)



13×

KMP over naive at
160,000 characters (1.05
vs 13.68 ms)

15.5×

Aho-Corasick over KMP x
k with 80 patterns

≈ 2×

Suffix array time per
doubling of n: $O(n \log n)$

≈ 4–6×

Levenshtein time per
doubling of both strings:
 $O(n \cdot m)$

Benchmark output

```

Java 25 · TextHack (DSA-3): benchmarks.Benchmark: exact and multi-pattern search

$ java -cp out benchmarks.Benchmark
=====
TextHack Benchmarks
=====
Best of 5 runs after 3 warm-up rounds.
Growth ratio compares each size against the previous one.

--- Exact single-pattern search ---
text length    naive      kmp      z      rabin-karp
20000          1644.0 us  1150.0 us  957.2 us  522.9 us
40000          3228.8 us  384.9 us  1793.5 us  868.3 us
80000          6775.1 us  525.8 us  511.9 us  1402.8 us
160000         13684.5 us 1050.9 us 1034.4 us  2844.6 us
Pattern a^40 b against a^n is the case that separates them:
naive rescans every alignment, the linear matchers do not.

--- Multi-pattern: Aho-Corasick versus repeated KMP ---
patterns      aho-corasick    kmp x k    speedup
5              5745.2 us      7499.5 us    1.3x
20             6538.3 us     30145.7 us   4.6x
80             7734.2 us     120252.9 us  15.5x
Aho-Corasick's cost is flat in the pattern count; running a
single-pattern matcher k times is not.

```

- $a^{40}b$ in a^n is naive search's worst case: every alignment rescans 40 characters
- Aho-Corasick cost is flat in the number of patterns

```

Java 25 · TextHack (DSA-3): benchmarks.Benchmark: suffix array and Levenshtein growth

$ java -cp out benchmarks.Benchmark (continued)
--- Suffix array construction ---
text length    time      growth
25000          3371.9 us  -
50000          3461.0 us  1.03x
100000         6692.9 us  1.93x
200000         15168.8 us 2.27x
O(n log n): doubling n should roughly double the time,
plus a little for the extra doubling round.

--- Levenshtein distance ---
string length   time      growth
250             413.0 us  -
500             904.4 us  2.19x
1000            5381.5 us 5.95x
2000            24587.8 us 4.57x
O(n*m): doubling both strings quadruples the work,
so a growth near 4x is the expected shape.

```

- Growth ratios, not absolute times, confirm each bound
- Best of 5 after 3 warm-ups on a laptop, so the smallest sizes are noisy

Test cases and results

Suite	Assertions	What it covers
StringAlgorithmTests	96	Edge cases, overlaps, pathological repeats, Unicode and NUL, 4,000 random cases vs naive
SuffixAndMultiPatternTests	75	Aho-Corasick nesting and duplicates, suffix and LCP arrays vs brute force
DpTests	61	Metric properties, OSA vs unrestricted Damerau, alignment scores and invariants
GraphTests	45	Three max-flow algorithms agree on 600 random networks; max-flow = min-cut
ApproximationAndRandomizedTests	86	Ratios vs brute-force optima, Carmichael numbers, sampling uniformity
EngineTests	89	Query parser, facade, citation flow, complexity registry
Total	452	0 failures · 0 compiler warnings

```

Java 25 · TextHack (DSA-3): sh run-tests.sh: all six suites (summary lines)

$ sh run-tests.sh > run-tests.log
$ grep -E "compiled|Test Summary|All suites" run-tests.log
compiled 39 source files
Test Summary: 96 passed, 0 failed
Test Summary: 75 passed, 0 failed
Test Summary: 61 passed, 0 failed
Test Summary: 45 passed, 0 failed
Test Summary: 86 passed, 0 failed
Test Summary: 89 passed, 0 failed
All suites passed

```

Re-run for this review with Java 25; the CI pipeline runs the same script on Java 21.

How the tests prove correctness

1

Randomised cross-validation

Efficient algorithms are checked against simple references on random inputs: 4,000 strings, 600 networks, 400 graphs.

2

Small alphabets on purpose

2–4 characters make overlaps and accidental matches common, which is where off-by-one errors surface.

3

Reproducible randomness

A hand-written seeded generator replaces `java.util.Random`, so a failing case can be replayed from its seed.

4

Theory as a test

Max flow equals min cut; approximation ratios hold against brute-force optima; a 40,000-trial check that sampling is uniform.

5

Adversarial inputs

Carmichael numbers, a 2,000-character repeat, text containing `\u0000`, patterns nested inside patterns.

```
Java 25 · TextHack (DSA-3): tests.GraphTests: the cross-validation sections (excerpt)

$ java -cp out tests.GraphTests (excerpt)
--- Known max-flow values ---
[PASS] CLRS example: three algorithms agree (23)
[PASS] CLRS network max flow is 23
[PASS] single edge: three algorithms agree (7)
[PASS] single edge flow is its capacity
[PASS] parallel: three algorithms agree (6)
[PASS] parallel paths sum their bottlenecks
[PASS] chain: three algorithms agree (3)
[PASS] chain is limited by its bottleneck
--- Ford-Fulkerson: the case that motivates Edmonds-Karp ---
[PASS] diamond: three algorithms agree (2000)
[PASS] diamond max flow is 2000
--- Three algorithms agree on random networks ---
[PASS] 600 random networks: all three algorithms agree
--- Flow conservation ---
[PASS] 300 networks conserve flow and match the reported value
--- Max-flow min-cut theorem ---
[PASS] 300 networks: min cut capacity equals max flow
--- Bipartite matching agrees with the max-flow reduction ---
[PASS] 400 bipartite graphs: Kuhn equals the max-flow reduction
Test Summary: 45 passed, 0 failed
```

Benefits of the proposed system



Typo-tolerant search

Damerau-Levenshtein finds "receive" from "recieve", which SQL LIKE cannot



Fast for many terms

Aho-Corasick checks every query term in one pass: 15.5× faster than KMP per term with 80 patterns



Guaranteed worst case

KMP and Z never degrade; no input makes them quadratic



Proven, not assumed

452 assertions, randomised cross-checks, and theorems verified as tests



No dependencies

Only a JDK to build and test; the same source runs in the backend



Explainable results

Complexity registry, alignments and min-cut citations shown to users

Real-world use cases

Enterprise document search

Keyword and fuzzy ranking in the platform's search

KMP · Aho-Corasick · OSA

Spell correction and autocomplete

Suggest the nearest dictionary words to a mistyped query

Levenshtein · Damerau

Plagiarism and duplicate detection

Local alignment isolates a shared paragraph between two documents

Smith-Waterman · suffix array

Research citation analysis

Count independent citation chains and find the citations they depend on

Dinic · min cut

Keyword filtering and security

Match thousands of blocked words or signatures in one pass

Aho-Corasick

Bioinformatics

The alignment algorithms were designed for DNA and protein sequences

Needleman-Wunsch

Issues encountered and how we solved them

Issue	Solution
java.util was barred from algorithm code	Wrote IntList, CharMap and a seeded PRNG; sorted suffixes with counting sort instead of a library sort
Z-algorithm's textbook sentinel breaks on text containing it	Compute the Z-array of the pattern only and match against the text: no sentinel, and $O(m)$ space
Rabin-Karp hash collisions give false matches at scale	Verify every hash hit character by character
$(a \cdot b) \bmod m$ overflows 64 bits in Miller-Rabin	Russian-peasant modular multiplication; BigInteger would replace the algorithm with a library call
"Interval scheduling" was listed under approximation, but it is solved exactly	Replaced with makespan scheduling, which is NP-hard: list scheduling $(2 - 1/m)$ and LPT $(4/3)$
Plain Levenshtein counts a swapped pair as 2 edits	Added Damerau-Levenshtein in both variants; the tests pin where they disagree
Aho-Corasick does not fit a one-pattern matcher interface	Gave it its own API returning which pattern matched, instead of one automaton per pattern

Scope for improvement

01

Indian-language corpus

Build an Indian-language Wikipedia corpus; needs dump selection and a licence review

02

Search at larger scale

The fuzzy scan scores up to 5,000 documents in memory; move candidate generation into a trigram index

03

Faster fuzzy lookup

BK-trees or Levenshtein automata to avoid comparing against every term

04

Linear-time suffix arrays

SA-IS construction in $O(n)$ instead of prefix doubling

05

More flow algorithms

Push-relabel max flow and weighted citation graphs

06

Parallel matching

Split large texts across threads for Aho-Corasick and KMP

CONCLUSION

A verified algorithm engine, running in a real product

20

algorithms, from scratch

452

assertions, 0 failures

22

complexity entries

13×

KMP vs naive, worst case

- Implemented string, DP, flow, approximation and randomized algorithms with no library shortcuts
- Proved them with cross-validation against references, adversarial inputs and theorems-as-tests
- Confirmed their complexity with benchmarks, and documented every bound
- Made the platform's search typo-tolerant and fast for many terms, and gave users a workbench to explore the algorithms

Research papers, books and resources

D. E. Knuth, J. H. Morris, V. R. Pratt, "Fast pattern matching in strings," SIAM J. Computing, 6(2), 1977.

A. V. Aho, M. J. Corasick, "Efficient string matching: an aid to bibliographic search," Comm. ACM, 18(6), 1975.

R. M. Karp, M. O. Rabin, "Efficient randomized pattern-matching algorithms," IBM J. Research and Development, 31(2), 1987.

U. Manber, G. Myers, "Suffix arrays: a new method for on-line string searches," SIAM J. Computing, 22(5), 1993.

T. Kasai et al., "Linear-time longest-common-prefix computation in suffix arrays and its applications," CPM, 2001.

V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, 10(8), 1966.

F. J. Damerau, "A technique for computer detection and correction of spelling errors," Comm. ACM, 7(3), 1964.

S. B. Needleman, C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," J. Molecular Biology, 48(3), 1970.

T. F. Smith, M. S. Waterman, "Identification of common molecular subsequences," J. Molecular Biology, 147(1), 1981.

L. R. Ford, D. R. Fulkerson, "Maximal flow through a network," Canadian J. Mathematics, 8, 1956.

J. Edmonds, R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," J. ACM, 19(2), 1972.

E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," Soviet Math. Doklady, 11, 1970.

H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, 2, 1955.

R. L. Graham, "Bounds for certain multiprocessing anomalies," Bell System Technical J., 45(9), 1966; "Bounds on multiprocessing timing anomalies," SIAM J. Applied Mathematics, 17(2), 1969.

G. L. Miller, "Riemann's hypothesis and tests for primality," J. Computer and System Sciences, 13(3), 1976; M. O. Rabin, "Probabilistic algorithm for testing primality," J. Number Theory, 12(1), 1980.

J. L. Carter, M. N. Wegman, "Universal classes of hash functions," J. Computer and System Sciences, 18(2), 1979.

J. S. Vitter, "Random sampling with a reservoir," ACM Trans. Mathematical Software, 11(1), 1985.

T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022. D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge Univ. Press, 1997.

```
TextHack> prime 7919
```

```
7919 is prime
```

Thank you

Questions and discussion

M. Sathya Krishna · Yashwanth · Aneeq · Abhinav Guide: Dr. P. Anitha

Department of Artificial Intelligence & Data Science · KLH University